

NutriApp

🕒 Created	May 10, 2026 11:50 AM
📁 Class	informatica

Relazione del Project Manager — Progetto NutriApp

Team di progetto:

- Barakat Ismail — Project Manager, Backend Developer, Data Engineer
- Donzelli Christian — Nutrition Expert, Information Researcher, Documentazione
- Maltese Emanuel — Frontend Developer, Wireframe
- Yu Serena — UX/UI Designer, Frontend Developer, Mockup

Indice

1. Introduzione al progetto
2. Il committente, i requisiti iniziali e l'analisi degli stakeholder
3. Analisi della concorrenza e posizionamento del prodotto
4. Scelte tecnologiche e motivazioni
5. Flutter e Dart — approfondimento tecnico completo
6. Architettura del progetto: struttura cartelle e flusso dati
7. Organizzazione del team, ruoli e metodologia Agile
8. Suddivisione del lavoro per sprint
9. Funzionalità implementate: core e aggiuntive
10. Le API alimentari: OpenFoodFacts e integrazione barcode
11. Qualità del codice: refactoring, commenti e GitLab
12. Valutazione del contributo di ciascun membro

13. Sfide incontrate e insegnamenti

14. Sviluppi futuri

15. Conclusione

1. Introduzione al progetto

NutriApp è un'applicazione mobile cross-platform progettata per il monitoraggio intelligente e dettagliato dell'alimentazione quotidiana. Consente di registrare i pasti, tracciare macro e micronutrienti, visualizzare progressi tramite grafici, impostare obiettivi personalizzati e ricevere avvisi in caso di superamento delle soglie nutrizionali.

Il progetto nasce da una richiesta formale della committente Prof.ssa Manuela Mileo (IIS Galilei, Crema) ed è stato sviluppato dal nostro team nel corso dell'anno scolastico, adottando una metodologia Agile con suddivisione in tre sprint incrementali.

Il lavoro svolto è stato, a mio parere, più che soddisfacente. In questo progetto ho ricoperto il ruolo di project manager, cercando di replicare le modalità organizzative tipiche del contesto lavorativo aziendale, forti dell'esperienza maturata durante il mio periodo di stage in Germania. Siamo riusciti a suddividere il lavoro in modo equilibrato e a collaborare efficacemente anche a distanza, sfruttando GitLab come strumento di versionamento condiviso. Abbiamo soddisfatto tutti i requisiti richiesti dalla committente e aggiunto funzionalità ulteriori non previste inizialmente.

2. Il committente, i requisiti iniziali e l'analisi degli stakeholder

2.1 La richiesta del committente

Il progetto NutriApp nasce da una richiesta formale della committente Prof.ssa Manuela Mileo, che ha espresso la necessità di sviluppare un'applicazione mobile non limitata al semplice conteggio calorico, ma uno strumento completo per il monitoraggio dettagliato dell'alimentazione, con una spiccata personalizzazione dei parametri da tracciare.

Dalla documentazione dei requisiti iniziali emergevano i seguenti punti principali:

- Monitoraggio dei valori nutrizionali assunti durante i pasti, con gestione di macro e micronutrienti
- Visualizzazione grafica dei progressi giornalieri, settimanali e mensili
- Personalizzazione degli obiettivi nutrizionali in base alle caratteristiche individuali dell'utente
- Sistema di notifiche e avvisi quando i valori si avvicinano o superano le soglie impostate
- Possibilità di registrare parametri fisici e conservare lo storico personale
- Compatibilità multiplatforma (in particolare iOS, essendo la committente utente iPhone)
- Interfaccia user-friendly
- Sicurezza dei dati utente e protezione della privacy (GDPR)
- Funzionalità di gestione del profilo utente
- Ricerca e consultazione di informazioni nutrizionali di prodotti alimentari
- Possibilità di salvare prodotti preferiti

2.2 Analisi degli stakeholder

Gli stakeholder identificati comprendono:

- **Prof.ssa Manuela Mileo** (committente): ruolo decisionale e strategico, fornisce linee guida e supervisione. Alto potere, alto interesse.
- **Utenti finali con esigenze nutrizionali specifiche** (es. diabetici, ipertesi): basso potere, alto interesse. Neutri/Supportivi.
- **Utenti orientati al miglioramento dello stile di vita**: basso potere, alto interesse. Neutri/Supportivi.
- **OpenFoodFacts** (fornitore API): dipendenza esterna da cui dipende la qualità dei dati nutrizionali.
- **Professionisti della nutrizione e strutture sanitarie**: potenziali utenti avanzati dello strumento.
- **Il team di sviluppo NutriApp**: esecutori del progetto.

2.3 Obiettivi formali (Project Charter)

In fase di avvio, abbiamo redatto il Project Charter, che definisce formalmente obiettivi, ambito, milestone e budget. I tre obiettivi principali erano:

1. **Lancio delle funzionalità core** entro fine febbraio 2026: inserimento manuale, scansione barcode, calcolo e visualizzazione del 100% dei macronutrienti e di almeno il 50% dei micronutrienti chiave.
2. **Sistema di notifiche push** entro metà marzo 2026: promemoria per la registrazione pasti e avvisi per il raggiungimento degli obiettivi giornalieri.
3. **Storico grafico delle statistiche** entro metà marzo 2026: sezione dedicata con almeno tre tipi di grafico (torta, barre, linee) per visualizzazioni giornaliere, settimanali e mensili.

Budget stimato: €124, comprendente la licenza Google Play Store (€25 una tantum) e la licenza Apple App Store (€99/anno). Le API alimentari pubbliche utilizzate (OpenFoodFacts e USDA FoodData Central) sono state impiegate nei rispettivi piani gratuiti.

3. Analisi della concorrenza e posizionamento del prodotto

Prima di definire le funzionalità da implementare, abbiamo condotto un'analisi delle applicazioni concorrenti che svolgono funzioni simili a NutriApp, con particolare attenzione a **Yazio** e **FatSecret**.

3.1 Yazio

Yazio è un'app di monitoraggio alimentare ampiamente diffusa, apprezzata per il design curato e la vasta banca dati di alimenti. Tra i suoi punti di forza figurano l'integrazione con wearable, i piani dietetici personalizzati e un'interfaccia visivamente rifinita. Tuttavia, le funzionalità avanzate sono quasi interamente bloccate dietro un abbonamento a pagamento, e il tracciamento dei micronutrienti è limitato nella versione gratuita. L'esperienza d'uso risulta spesso sovraccarica di contenuti promozionali e suggerimenti non richiesti.

3.2 FatSecret

FatSecret punta su una banca dati alimentare molto ampia e su un sistema di community che consente agli utenti di aggiungere alimenti. È gratuita e completa nei dati calorici, ma presenta un'interfaccia datata, una navigazione

poco intuitiva e un sistema di tracciamento dei micronutrienti incompleto e difficile da leggere.

3.3 Il nostro posizionamento

Dall'analisi comparativa abbiamo ricavato una visione chiara di cosa volevamo evitare e cosa volevamo migliorare. NutriApp si propone come:

- Completamente **gratuita**, senza funzionalità bloccate da paywall
 - Focalizzata su una **visualizzazione chiara e immediata** dei dati, senza rumore visivo
 - Capace di tracciare **macro e micronutrienti in modo completo**, inclusi elettroliti, fibre e acqua
 - Dotata di **scansione barcode** integrata fin dall'inizio, non come funzione premium
 - Progettata con **soglie personalizzabili** per condizioni cliniche specifiche (diabete, ipertensione, ecc.)
 - Orientata a un **uso reale quotidiano**, con ricette personalizzate e storico dati
-

4. Scelte tecnologiche e motivazioni

4.1 Perché un'app mobile e non una webapp

Una delle prime e più importanti decisioni strategiche del progetto è stata rinunciare a una webapp in favore di un'applicazione mobile nativa. Questa scelta è stata motivata da diversi fattori:

- **Praticità d'uso:** un'app mobile è sempre con l'utente, accessibile in tempo reale durante i pasti, al supermercato o in palestra. Una webapp richiederebbe l'apertura del browser e un'esperienza meno fluida.
- **Gestione dell'autenticazione:** le app mobile offrono sistemi di sessione persistente più robusti, con salvataggio locale delle credenziali e login automatico.
- **Accesso all'hardware del dispositivo:** la scansione del codice a barre richiede l'accesso diretto alla fotocamera, molto più semplice e performante in un contesto nativo rispetto al browser.

- **Richiesta esplicita della committente:** essendo utente iPhone, la Prof.ssa Mileo ha esplicitamente richiesto la compatibilità con iOS, il che ha orientato la scelta verso un framework cross-platform mobile.
- **Approccio architetturale moderno:** lo sviluppo di app mobile rappresenta oggi lo standard per prodotti consumer rivolti all'utente finale.

4.2 Lo stack tecnologico completo

Tecnologia	Ruolo nel progetto	Motivazione
Flutter + Dart	Frontend cross-platform	Un unico codebase, UI reattiva, sviluppo rapido
Riverpod	State management	Separazione chiara tra UI e logica, stato condiviso tipizzato
HTTP (<code>package:http</code>)	Comunicazione client-server	Chiamate REST verso endpoint PHP e API esterne
PHP	Backend applicativo	Gestione CRUD, autenticazione e business logic (in <code>fileDatabase/</code>)
MySQL/MariaDB	Persistenza dati	Struttura relazionale per utenti, pasti, ricette e obiettivi
OpenFoodFacts API	Fonte dati internazionale	Dataset ampio e gratuito, utile per i barcode dei prodotti
USDA FoodData API	Fonte di fallback	Copertura aggiuntiva quando i dati OFF sono assenti
Database CREA	Fonte italiana prioritaria	Migliore pertinenza per query testuali in italiano
<code>barcode_scan2</code>	Scansione barcode	Integrazione rapida con fotocamera e lettura EAN
<code>local_notifications</code>	Notifiche pianificate	Promemoria pasti/acqua direttamente sul dispositivo
GitLab	Versionamento	Branching, merge e tracciabilità del lavoro di team

5. Flutter e Dart — approfondimento tecnico completo

5.1 Dart: il linguaggio

Dart è un linguaggio di programmazione sviluppato da Google, progettato per essere **semplice, fortemente tipizzato, compilato e orientato agli oggetti**.

Prima di iniziare lo sviluppo, durante le vacanze natalizie, tutto il team ha seguito un tutorial intensivo di diverse ore sulle basi di Dart e Flutter, ed ho personalmente redatto un manuale di riferimento nella cartella della documentazione del progetto.

Le caratteristiche principali di Dart che abbiamo utilizzato nel progetto:

Tipizzazione forte e null safety

Dart implementa la **null safety** a partire dalla versione 2.12. Questo significa che, a meno che non lo si dichiari esplicitamente con `?`, nessuna variabile può essere null. Questo elimina interi classi di errori a runtime che sarebbero difficili da individuare.

```
String nome = "Ismail";           // non può essere null
String? cognome = null;           // può essere null, dichiarato esplicitamente

// Per usare cognome in modo sicuro:
int lunghezza = cognome?.length ?? 0;
```

Tipizzazione statica con inferenza

Dart è staticamente tipizzato, ma il compilatore è in grado di inferire il tipo automaticamente grazie alla keyword `var` o `final`:

```
var calorie = 250;                // inferito come int
final nome = "NutriApp";          // inferito come String, non modificabile
const pi = 3.14;                 // costante compilata, immutabile e ottimizzata
```

Programmazione asincrona: `async` / `await` e `Future`

In NutriApp, quasi tutte le chiamate al backend PHP avvengono in modo asincrono. Dart gestisce questo con il sistema `Future` e le keyword `async` / `await`, che rendono il codice asincrono leggibile come se fosse sincrono:

```
Future<List<Alimento>> fetchAlimentiGiornata(String data) async {
  final response = await http.get(Uri.parse('$baseUrl/get_alimenti.php?data=$data'));
  if (response.statusCode == 200) {
    final List<dynamic> data = json.decode(response.body);
    return data.map((json) => Alimento.fromJson(json)).toList();
  } else {
    throw Exception('Errore nel caricamento degli alimenti');
  }
}
```

Il costrutto `await` sospende l'esecuzione della funzione corrente senza bloccare il thread principale dell'interfaccia, garantendo un'app sempre reattiva.

Stream

Oltre ai `Future` (che restituiscono un singolo valore in modo asincrono), Dart supporta gli `Stream`, che emettono una sequenza di valori nel tempo. Nel progetto li abbiamo utilizzati per l'aggiornamento in tempo reale dei grafici tramite Riverpod.

Classi, costruttori e modelli dati

Dart è pienamente orientato agli oggetti. Ogni entità del dominio di NutriApp (alimento, ricetta, utente, pasto) è rappresentata da una classe con costruttori named e metodi di serializzazione/deserializzazione JSON:

```
class Alimento {
  final String nome;
  final double calorie;
  final double proteine;
  final double carboidrati;
  final double grassi;
```



```

Alimento({
  required this.nome,
  required this.calorie,
  required this.proteine,
  required this.carboidrati,
  required this.grassi,
});

factory Alimento.fromJson(Map<String, dynamic> json) {
  return Alimento(
    nome: json['nome'] as String,
    calorie: (json['calorie'] as num).toDouble(),
    proteine: (json['proteine'] as num).toDouble(),
    carboidrati: (json['carboidrati'] as num).toDouble(),
    grassi: (json['grassi'] as num).toDouble(),
  );
}

Map<String, dynamic> toJson() => {
  'nome': nome,
  'calorie': calorie,
  'proteine': proteine,
  'carboidrati': carboidrati,
  'grassi': grassi,
};
}

```

Il pattern `factory constructor` con `fromJson` è fondamentale in NutriApp perché tutta la comunicazione col backend avviene in formato JSON.

Mixins e extension

Dart supporta i **mixin**, che consentono di aggiungere funzionalità a una classe senza ereditarietà multipla, e le **extension**, che permettono di aggiungere metodi a classi esistenti (anche di librerie esterne) senza modificarle:

```

extension StringCapitalize on String {
  String capitalize() => '${this[0].toUpperCase()}${substri

```

```

ng(1)}';
}

// Utilizzo:
print('nutriapp'.capitalize()); // "Nutriapp"

```

5.2 Flutter: architettura e sistema di widget

Flutter è il framework open source sviluppato da Google che utilizza Dart come linguaggio di programmazione. La sua caratteristica distintiva è il **motore di rendering proprietario (Skia / Impeller)**, che disegna ogni pixel dell'interfaccia in modo indipendente dal sistema operativo sottostante. Questo significa che un'app Flutter ha lo stesso aspetto e le stesse prestazioni su Android, iOS, Web e Desktop, senza dipendere dai componenti UI nativi di ogni piattaforma.

Il concetto di widget

In Flutter, **tutto è un widget**. Un widget è un'unità immutabile che descrive una porzione dell'interfaccia utente. L'intera UI di NutriApp è costruita componendo widget in una struttura ad albero chiamata **widget tree**.

```

// Esempio semplificato di una card nutriente in NutriApp
class CardNutriente extends StatelessWidget {
  final String nome;
  final double valore;
  final double massimo;
  final Color colore;

  const CardNutriente({
    super.key,
    required this.nome,
    required this.valore,
    required this.massimo,
    required this.colore,
  });

  @override
  Widget build(BuildContext context) {
    return Card(

```

```

        child: Padding(
          padding: const EdgeInsets.all(16.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(nome, style: const TextStyle(fontWeight: FontWeight.bold)),
              const SizedBox(height: 8),
              LinearProgressIndicator(
                value: valore / massimo,
                color: colore,
                backgroundColor: colore.withOpacity(0.2),
              ),
              Text('${valore.toStringAsFixed(1)} / ${massimo.toStringAsFixed(0)} g'),
            ],
          ),
        ),
      );
    }
  }
}

```

StatelessWidget vs StatefulWidget

Flutter distingue due tipi fondamentali di widget:

- **StatelessWidget**: descrive una porzione di UI che non cambia nel tempo. È immutabile: una volta costruito, non si aggiorna. Utilizzato per componenti statici come loghi, titoli, card di sola lettura.
- **StatefulWidget**: descrive una porzione di UI che può cambiare nel tempo in risposta a eventi (input utente, dati dal server, timer). Internamente mantiene un oggetto `State` che può essere aggiornato tramite `setState()`.

```

class ContatorePasti extends StatefulWidget {
  const ContatorePasti({super.key});

  @override
  State<ContatorePasti> createState() => _ContatorePastiState();
}

```

```

}

class _ContatorePastiState extends State<ContatorePasti> {
  int _numeroPasti = 0;

  void _aggiungiPasto() {
    setState(() {
      _numeroPasti++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Pasti registrati oggi: $_numeroPasti'),
        ElevatedButton(
          onPressed: _aggiungiPasto,
          child: const Text('Aggiungi pasto'),
        ),
      ],
    );
  }
}

```

Ogni volta che si chiama `setState()`, Flutter riesegue il metodo `build()` del widget e aggiorna solo le parti dell'interfaccia che sono effettivamente cambiate, grazie al meccanismo del **diffing dell'albero dei widget**.

Il ciclo di vita di un widget StatefulWidget

Il ciclo di vita completo di uno StatefulWidget comprende:

1. `createState()` : viene creato l'oggetto State
2. `initState()` : chiamato una sola volta alla creazione. Ideale per inizializzare dati, avviare chiamate HTTP, sottoscrivere a stream.
3. `didChangeDependencies()` : chiamato dopo initState e ogni volta che le dipendenze del widget cambiano.

4. `build()` : costruisce l'albero dei widget. Viene rieseguito ogni volta che lo stato cambia.
5. `didUpdateWidget()` : chiamato quando il widget padre ricostruisce e passa nuove proprietà.
6. `dispose()` : chiamato quando il widget viene rimosso dall'albero. Fondamentale per chiudere controller, stream e listener e prevenire memory leak.

```
@override
void initState() {
  super.initState();
  _caricaDatiGiornata(); // chiamata HTTP al backend
}

@override
void dispose() {
  _scrollController.dispose(); // prevenzione memory leak
  super.dispose();
}
```

Layout e composizione

Flutter utilizza un sistema di layout basato su vincoli. Ogni widget riceve i vincoli (dimensioni min/max) dal widget padre e decide come occupare lo spazio disponibile. I widget di layout principali che abbiamo usato in NutriApp:

- `Column` / `Row` : dispone i figli verticalmente o orizzontalmente
- `Stack` : sovrappone widget (usato per le animazioni di scansione barcode)
- `ListView` / `GridView` : liste scorrevoli (usate per la lista degli alimenti e le ricette)
- `Padding` / `SizeBox` / `Expanded` : controllo preciso di spaziatura e dimensioni
- `SingleChildScrollView` : aggiunge scorrimento a contenuti che potrebbero non stare nello schermo
- `SafeArea` : garantisce che il contenuto non venga coperto da notch, barre di stato e gesture bar

5.3 State management con Riverpod

Per applicazioni complesse come NutriApp, gestire lo stato solo con `setState()` diventa rapidamente ingestibile, perché lo stato deve essere condiviso tra schermate diverse e aggiornato in risposta a eventi remoti. Abbiamo utilizzato **Riverpod**, uno dei package di state management più moderni e tipizzati per Flutter.

Riverpod si basa sul concetto di **provider**: un'unità di stato globale, accessibile da qualsiasi widget nell'albero, che si aggiorna automaticamente quando i dati cambiano.

```
// Definizione di un provider per i dati nutrizionali della giornata
final nutrientiGiornataProvider = FutureProvider.family<RiepilogoGiornata, String>(
  (ref, data) async {
    final apiService = ref.read(apiServiceProvider);
    return await apiService.fetchNutrientiGiornata(data);
  },
);

// Utilizzo nel widget
class HomeScreen extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final nutrientiAsync = ref.watch(nutrientiGiornataProvider(oggi));

    return nutrientiAsync.when(
      loading: () => const CircularProgressIndicator(),
      error: (err, _) => Text('Errore: $err'),
      data: (nutrienti) => GraficoNutriente(dati: nutrienti),
    );
  }
}
```

Il vantaggio principale di Riverpod in NutriApp è che i grafici e i riepiloghi si aggiornano automaticamente ogni volta che l'utente registra un nuovo alimento,

senza dover gestire manualmente le notifiche di aggiornamento tra schermate diverse.

5.4 Packages principali utilizzati nel progetto

Flutter dispone di un ecosistema ricco di package open source distribuiti tramite **pub.dev**. Quelli principali utilizzati in NutriApp:

- **http**: client HTTP per le chiamate al backend PHP. Gestisce GET, POST, PUT, DELETE con supporto completo agli header e al corpo della richiesta in JSON.
 - **barcode_scan2**: accesso alla fotocamera del dispositivo per la lettura di codici a barre EAN. Restituisce il codice scansionato come stringa, che viene poi inviata al backend per la ricerca su OpenFoodFacts.
 - **fl_chart**: libreria per la creazione di grafici a torta (macronutrienti), a barre (micronutrienti e calorie giornaliere) e lineari (andamento settimanale e mensile).
 - **riverpod**: state management reattivo e tipizzato, utilizzato per la condivisione dello stato tra le schermate.
 - **shared_preferences**: storage locale key-value per il salvataggio della sessione utente (token di autenticazione, preferenze).
 - **pdf** e **printing**: generazione e stampa di report nutrizionali in formato PDF direttamente dall'app.
 - **intl**: internazionalizzazione e formattazione di date, numeri e valute (es. visualizzazione delle date nel diario alimentare).
-

5.5 Compilazione cross-platform: come Flutter gira su Android e iOS

La domanda più frequente su Flutter è: come riesce a produrre app native per piattaforme diverse da un singolo codebase?

Flutter non utilizza una WebView né i componenti UI nativi del sistema operativo (come fa React Native). Invece, disegna ogni pixel dell'interfaccia direttamente sul canvas del dispositivo tramite il motore grafico **Skia** (o il più recente **Impeller** su iOS). Questo significa che l'app ha lo stesso aspetto esatto su qualsiasi dispositivo, indipendentemente dalla versione del sistema operativo.

Il processo di compilazione è il seguente:

- **Android:** il codice Dart viene compilato in codice macchina nativo ARM tramite **AOT compilation** (Ahead-of-Time). Il risultato è un file APK o AAB che viene distribuito tramite il Google Play Store.
- **iOS:** analogamente, il codice Dart viene compilato in codice macchina ARM nativo. Il risultato è un'app .ipa distribuibile tramite l'Apple App Store. Richiede un Mac con Xcode per la compilazione finale e la firma del codice.

Per lo sviluppo, Flutter utilizza invece la **JIT compilation** (Just-in-Time), che permette il **Hot Reload**: una funzionalità che aggiorna l'interfaccia in meno di un secondo senza riavviare l'app, mantenendo lo stato corrente. Questo ha accelerato enormemente il nostro ciclo di sviluppo.

Per testare la versione iOS senza un account sviluppatore Apple a pagamento, abbiamo utilizzato **AltServer + AltStore**, che permette di installare l'app firmata localmente su un dispositivo iPhone reale.

6. Architettura del progetto: struttura cartelle e flusso dati

6.1 Struttura delle cartelle

Il progetto è organizzato con una struttura modulare, pensata per facilitare la manutenibilità, la divisione del lavoro e la scalabilità futura:

```
nutriapp/
├── lib/
│   ├── screens/           # Schermate principali dell'app
│   │   ├── home_screen.dart
│   │   ├── calendar_screen.dart
│   │   ├── add_food_screen.dart
│   │   ├── recipes_screen.dart
│   │   ├── charts_screen.dart
│   │   ├── settings_screen.dart
│   │   └── login_screen.dart
│   ├── models/           # Modelli dati (Alimento, Ricetta, Utente, Riepilogo)
│   ├── services/         # Comunicazione con il backend (ApiServices)
│   ├── providers/        # State management con Riverpod
│   ├── domain/           # Logica di business condivisa
│   └── widgets/          # Componenti grafici riutilizzabili
├── fileDatabase/         # Script PHP e file SQL del backend
├── android/              # Layer di compilazione Android
├── ios/                  # Layer di compilazione iOS
├── web/                  # Layer compilazione Web (predisposto)
├── windows/              # Layer compilazione Windows (predisposto)
└── Documentazione/      # Documenti LaTeX, appunti, manuale Dart/Flutter
```


6.2 Flusso dati end-to-end

Il flusso dati principale dell'applicazione segue un percorso strutturato e ben separato tra frontend e backend:

1. L'utente interagisce con una schermata Flutter (es. registra un alimento)
2. La schermata usa un model o un provider Riverpod per organizzare i dati
3. Il servizio `ApiServices` invia una richiesta HTTP POST al backend PHP
4. Il file PHP riceve i dati, esegue le operazioni necessarie e interroga il database MySQL/MariaDB
5. Il database salva o restituisce i dati richiesti
6. La risposta viene elaborata dal backend e restituita all'app in formato JSON
7. Flutter decodifica il JSON, aggiorna i provider Riverpod e l'interfaccia si aggiorna automaticamente

Questa separazione netta tra logica client e server rende più semplice mantenere il progetto nel tempo e implementare aggiornamenti senza modificare la struttura base.

7. Organizzazione del team, ruoli e metodologia Agile

7.1 Metodologia Agile e sprint

Lo sviluppo è stato condotto adottando la **metodologia Agile**, con il lavoro suddiviso in tre sprint incrementali, ciascuno focalizzato sull'implementazione di funzionalità specifiche e sulla validazione continua dei risultati.

Le attività sono state organizzate sulla base di un **Product Backlog** strutturato in User Stories, formulate secondo il modello: *"Come [ruolo] vorrei [azione] per [scopo]"*.

Le funzionalità sono state classificate per priorità secondo il modello **MoSCoW**:

- **MUST** (Priorità 1): interfaccia intuitiva, inserimento manuale, scansione barcode, calcolo totali giornalieri, notifiche push
- **SHOULD** (Priorità 2): registrazione account, ricette personalizzate, soglie nutrizionali, grafici giornalieri
- **MAY** (Priorità 3): report periodici settimanali/mensili, tracciamento parametri fisici

7.2 Ruoli nel team

Membro	Ruolo principale	Competenze specifiche
Barakat Ismail	Project Manager, Backend Dev, Data Engineer	Backend PHP/MySQL, API integration, gestione team, testing, branching
Donzelli Christian	Nutrition Expert, Documentazione	Ricerca scientifica, redazione LaTeX, gestione privacy/GDPR
Maltese Emanuel	Frontend Developer	Flutter, schermate funzionali, wireframe
Yu Serena	UX/UI Designer, Frontend Dev	Figma, Balsamiq, mockup, identità visiva, schermate UI

Come project manager, ho gestito la suddivisione dei compiti, l'assegnazione delle attività a inizio sprint, il monitoraggio dell'avanzamento, i merge su GitLab e l'assemblaggio finale del prodotto. L'esperienza maturata durante il mio periodo in Germania mi ha permesso di applicare in modo naturale le pratiche organizzative tipiche di un team di sviluppo professionale, dalla gestione dei branch al code review sistematico.

8. Suddivisione del lavoro per sprint

8.1 Sprint 1 — Analisi, progettazione e formazione

Il primo sprint è stato dedicato interamente alla fase preliminare di definizione del progetto. L'obiettivo era allineare il team su cosa costruire prima di entrare nello sviluppo tecnico.

Attività svolte:

- Raccolta e studio dei requisiti della committente
- Analisi degli stakeholder e redazione del Project Charter
- Preparazione del Product Backlog e delle User Stories
- Realizzazione di wireframe (Barakat e Maltese) su Figma/Balsamiq
- Realizzazione di mockup (Donzelli e Serena) con bozze dell'icona e identità visiva
- Raccolta e compilazione delle informazioni nutrizionali di base degli alimenti più comuni (Donzelli, ~14h)

Formazione tecnica:

Durante le vacanze natalizie, tutto il team si è accordato per seguire un tutorial intensivo di diverse ore sulle basi di Dart e di Flutter. Trattandosi di un linguaggio completamente nuovo per tutti, era fondamentale avere un punto di partenza comune. Ho poi redatto personalmente un **manuale di riferimento su Dart e Flutter** nella cartella della documentazione, con esempi pratici e spiegazioni delle principali caratteristiche del linguaggio, consultabile da tutti i membri durante lo sviluppo.

Attività	Ore	Chi
Progettazione wireframe e revisione mockup	24h	Barakat Ismail
Progettazione mockup	22h	Yu Serena
Supporto wireframe e mockup	22h	Maltese Emanuel
Ricerca nutrizionale e documentazione base	14h	Donzelli Christian

8.2 Sprint 2 — Sviluppo della base funzionale

Nel secondo sprint abbiamo costruito la base funzionante dell'applicazione. Siamo partiti a pieno ritmo: abbiamo redatto lo sprint backlog e portato a termine tutto ciò che ci eravamo prefissati.

User Story 1 — Registrazione manuale dei valori nutrizionali

"Come Utente vorrei registrare manualmente i valori nutrizionali di un alimento (macro, micro, acqua) per monitorare la mia alimentazione."

Attività	Ore	Chi
Backend e gestione dei dati	8h	Barakat Ismail
Documentazione LaTeX (sezioni 1-2, introduzione e strategia)	5h	Donzelli Christian
Interfaccia grafica inserimento manuale	7h	Maltese Emanuel
Interfaccia homepage e pre-inserimento nutrienti	6h	Yu Serena

User Story 2 — Calcolo automatico dei totali giornalieri

"Come Sistema vorrei calcolare automaticamente i totali giornalieri dei nutrienti."

Attività	Ore	Chi
Backend e gestione dei dati	7h	Barakat Ismail

Attività	Ore	Chi
Documentazione LaTeX (sezione 4, fonti dati e ricerca scientifica)	6h	Donzelli Christian
Interfaccia dettagli giorno	6h	Maltese Emanuel
Interfaccia dettagli dei pasti	7h	Yu Serena

User Story 3 — Registrazione ricette personalizzate

"Come Utente vorrei poter registrare le mie ricette per non doverle reinserire."

Attività	Ore	Chi
Backend e gestione dei dati	7h	Barakat Ismail
Documentazione LaTeX (sezione 4, ricerca scientifica)	4h	Donzelli Christian
Interfaccia salvataggio ricetta	5h	Maltese Emanuel
Interfaccia aggiunta ingredienti ricetta	6h	Yu Serena

User Story 4 — Salvataggio ricette lato sistema

"Come Sistema vorrei salvare le ricette inserite dall'utente per facilitare il calcolo delle calorie."

Attività	Ore	Chi
Backend e gestione dei dati	7h	Barakat Ismail
Documentazione LaTeX (fact-checking, riorganizzazione, immagini)	5h	Donzelli Christian

Avendo concluso le funzionalità principali in anticipo rispetto alle stime, abbiamo anche avviato la login page, le impostazioni e gettato le basi per le schermate secondarie.

8.3 Sprint 3 — Estensione delle funzionalità e rifinitura

Il terzo sprint è stato il più intenso. Abbiamo completato il progetto nella sua interezza, implementando le funzionalità avanzate, resolvendo tutte le incongruenze e rifinendo l'intera applicazione.

User Story 1 — Scansione barcode

"Come Utente vorrei poter scansionare il barcode di un alimento per inserire i valori nutrizionali in maniera semi-automatica."

Attività	Ore	Chi
Integrazione API OpenFoodFacts	5h	Barakat Ismail
Documentazione LaTeX: privacy come documento esterno	2.5h	Donzelli Christian
Interfaccia fotocamera e animazione di scansione	5h	Maltese Emanuel
Interfaccia loading e messaggi di errore	4h	Yu Serena

User Story 2 — Registrazione e login

"Come Utente vorrei registrare un account ed accedervi per poter salvare i miei progressi."

Attività	Ore	Chi
Tabelle DB utenti, script PHP login/registrazione, hash password	4h	Barakat Ismail
Documentazione LaTeX: parte 5, informativa legale e distribuzione	2.5h	Donzelli Christian
Interfaccia registrazione e login (form, validazione password)	5h	Maltese Emanuel
Interfaccia dettagli profilo	5h	Yu Serena

User Story 3 — Soglie nutrizionali personalizzate

"Come Utente vorrei impostare soglie personalizzate (es. max zuccheri) per gestire le mie esigenze specifiche (es. diabete)."

Attività	Ore	Chi
Aggiornamento schema DB per obiettivi/soglie e API aggiornamento	4.5h	Barakat Ismail
Documentazione LaTeX: porting documenti in LaTeX	2.5h	Donzelli Christian
Interfaccia "I miei obiettivi" (soglie peso, calorie, zuccheri)	4h	Maltese Emanuel
Query SQL per confronto macro giornalieri con limiti utente	4h	Barakat Ismail
Widget Alert/Popup superamento soglia nella Home	2h	Maltese Emanuel
Progress bar dinamica (diventa rossa se i valori sono eccessivi)	4h	Yu Serena

User Story 4 — Grafici e report

"Come Utente vorrei visualizzare grafici giornalieri dei nutrienti per comprendere immediatamente i miei progressi."

Attività	Ore	Chi
Endpoint PHP per fetch dati aggregati giornalieri	4h	Barakat Ismail
Script PHP per formattazione dati JSON per grafici	4h	Barakat Ismail
Interfaccia grafici a torta (macronutrienti principali)	4h	Maltese Emanuel
Interfaccia grafici a barre (micronutrienti e calorie)	4h	Yu Serena
Setup pacchetto grafico e widget grafici base	3h	Maltese Emanuel
Gestione stato con Riverpod per aggiornamento real-time grafici	3h	Yu Serena

User Story 5 — Notifiche push

"Come Sistema vorrei inviare una notifica push immediata all'utente per avvisarlo del superamento della soglia."

Attività	Ore	Chi
Configurazione notifiche locali pianificate con <code>flutter_local_notifications</code>	4h	Barakat Ismail
UI richiesta permessi notifiche iOS/Android	3h	Maltese Emanuel
Interfaccia "Storico Notifiche"	3h	Yu Serena

User Story 6 — Report periodici e parametri fisici

Attività	Ore	Chi
Endpoint PHP con query di aggregazione settimanali/mensili	4h	Barakat Ismail
Interfaccia report giornaliero	6h	Maltese Emanuel
Interfaccia report settimanale (trend e confronti obiettivi)	4h	Maltese Emanuel
Interfaccia report mensile (calendario storico)	6h	Yu Serena
Creazione tabella DB parametri fisici e API	4h	Barakat Ismail

Attività	Ore	Chi
Form inserimento misurazioni fisiche (es. peso)	3h	Maltese Emanuel
Storico misurazioni fisiche (lista e grafico andamento peso)	4h	Yu Serena

9. Funzionalità implementate: core e aggiuntive

9.1 Funzionalità core (requisiti della committente rispettati)

Tutti i requisiti base indicati dalla committente sono stati pienamente rispettati:

- Monitoraggio completo dell'assunzione alimentare per pasto e per giorno, con gestione di macro e micronutrienti, fibre, acqua ed elettroliti
- Impostazione di obiettivi nutrizionali personalizzati in base alle caratteristiche individuali dell'utente
- Visualizzazione dei progressi tramite schermate di riepilogo e grafici giornalieri, settimanali e mensili
- Sistema di avvisi quando i valori si avvicinano o superano le soglie impostate
- Salvataggio persistente dei dati utente e delle registrazioni alimentari
- Interfaccia user-friendly con navigazione semplice e intuitiva
- Gestione del profilo utente con dati personali e obiettivi
- Ricerca e consultazione di informazioni nutrizionali di prodotti alimentari
- Sicurezza dei dati e protezione della privacy conforme al GDPR
- Compatibilità multiplatforma garantita da Flutter

9.2 Funzionalità aggiuntive (oltre i requisiti iniziali)

Oltre alle richieste iniziali, il team ha integrato:

- **Scansione del codice a barre** dei prodotti, con recupero automatico dei dati nutrizionali tramite i vari DB
- **Gestione completa delle ricette personalizzate**, con ingredienti, porzioni, note e salvataggio nel backend
- **Backend dedicato** in PHP e database relazionale MySQL/MariaDB

- **Esportazione dei report** nutrizionali in formato PDF (consigliato dlla prof di motoria)
 - **Modalità scura**
 - **Personalizzazione avanzata del profilo**
 - **Guide per l'utente** (tutorial in-app)
 - **Privacy policy** integrata nell'applicazione
 - Lingue app
 - Social login e OTP autenticazione
-

10. Strategia di Ricerca Alimentare Multi-Fonte

Il cuore tecnologico di NutriApp risiede nel suo motore di ricerca ibrido. A differenza delle soluzioni standard che si affidano a un unico provider, NutriApp implementa una **strategia multi-fonte** per massimizzare la copertura del database e la precisione dei valori nutrizionali.

10.1 Flusso di Scansione Barcode

Il riconoscimento tramite codice a barre segue una logica gerarchica di "failover" per garantire tempi di risposta rapidi e dati accurati:

1. **Input:** L'utente attiva la fotocamera tramite il package `barcode_scan2`.
 2. **Esecuzione:** Viene invocato il metodo `ApiServices.fetchProductByBarcode()`.
 3. **Cascata di interrogazione (Sequential Fallback):**
 - **DB Locale (PHP):** Controllo immediato nel database interno (`get_product_by_barcode.php`) per prodotti già validati.
 - **OpenFoodFacts:** Interrogazione del database internazionale per prodotti confezionati (EAN/UPC).
 - **USDA FoodData Central:** Fallback finale per prodotti di importazione o materie prime.
 4. **Output:** I dati grezzi vengono **normalizzati** in un modello dati interno unico prima della visualizzazione.
-

10.2 Architettura della Ricerca Libera

Per la ricerca testuale, NutriApp adotta una **pipeline asincrona a due livelli**, progettata per bilanciare la velocità di risposta con la profondità del database.

Fase 1: Local-First (Latency Zero)

L'app interroga istantaneamente il database locale dell'utente. Questo permette di:

- Mostrare immediatamente gli **alimenti personalizzati** creati dall'utente.
- Dare priorità allo **storico dei consumi**, rendendo la ricerca più "intelligente" in base alle abitudini passate.

Fase 2: Multi-Source Parallela

In background, il sistema lancia tre richieste asincrone simultanee verso i provider esterni:

1. **CREA**: Fonte prioritaria per la pertinenza con la lingua italiana e la dieta mediterranea.
2. **OpenFoodFacts**: Utilizzata per attingere al vasto catalogo dei prodotti confezionati globali.
3. **USDA**: Interrogata come database scientifico di riferimento per gli alimenti generici.

10.2.1 Logica di Unificazione e Ranking

Una volta ricevuti i dati, il sistema applica un algoritmo di **ordinamento ibrido** per presentare i risultati in un'unica lista coerente, basandosi su tre parametri:

- **Pertinenza Testuale**: Corrispondenza tra la query inserita e il nome dell'alimento (token matching).
- **Gerarchia delle Fonti**: I risultati provenienti dal database **CREA** ricevono un punteggio di priorità più alto nel contesto italiano.
- **Soglia di Qualità**: Il ranking penalizza i risultati che presentano valori nutrizionali incompleti (es. record privi di macronutrienti), favorendo le voci più dettagliate.

Risultato: L'utente visualizza una lista unificata e deduplicata dove i risultati più accurati e vicini alle proprie abitudini compaiono sempre nelle prime posizioni.

10.3 Analisi Tecnica e Vantaggi

Caratteristica	Vantaggio per l'utente
Riduzione Lacune	Se una fonte manca di micro-nutrienti (es. fibre o potassio), il sistema attinge dalle altre.
Pertinenza Locale	L'uso prioritario dei dati CREA garantisce che alimenti tipici della dieta mediterranea siano accurati.
Scalabilità	L'architettura modulare permette di aggiungere nuove API in futuro senza riscrivere la logica UI.

10.4 Limitazioni Note e Sfide

- **Eterogeneità dei dati:** La precisione dei micronutrienti varia sensibilmente tra prodotti industriali e alimenti generici.
- **Normalizzazione:** È necessaria una manutenzione costante dei mapper per uniformare le unità di misura (es. mg vs µg) provenienti da diverse API.
- **Dati Mancanti:** Alcuni prodotti confezionati riportano in etichetta solo i dati obbligatori per legge, limitando la profondità dell'analisi nutrizionale.

11. Qualità del codice: refactoring, commenti e GitLab

11.1 Refactoring e pulizia del codice

Fin dall'inizio del progetto ho prestato moltissima attenzione al refactoring e alla pulizia del codice, occupandomene spesso personalmente. L'obiettivo era rendere il codice manutenibile nel tempo e comprensibile a chiunque, anche a chi non ha scritto quella parte specifica.

Le pratiche adottate sistematicamente:

- **Estrazione di widget riutilizzabili:** ogni componente grafico che veniva utilizzato in più schermate è stato estratto in un widget dedicato nella cartella `lib/widgets/`, evitando duplicazione del codice.
- **Naming semantico:** tutte le variabili, le funzioni e le classi sono state nominate in modo da rendere esplicito il loro scopo, senza richiedere commenti aggiuntivi per capire cosa fanno.
- **Separazione delle responsabilità:** la logica di business è rimasta nei provider e nei service, le schermate si occupano solo della presentazione, i model gestiscono la struttura dei dati.

- **Refactoring delle schermate:** nel terzo sprint ho eseguito un refactoring sistematico delle varie pagine, migliorandone la grafica, modernizzando i componenti e standardizzando i pattern di comunicazione col backend.

11.2 Commenti nel codice

Il team è stato molto attento a lasciare commenti chiari e significativi nel codice, così da facilitare la comprensione reciproca durante il lavoro in parallelo e in remoto. Un buon commento in NutriApp non spiega *cosa* fa il codice (che deve essere già chiaro dalla lettura), ma *perché* è stato scritto in quel modo, o segnala limitazioni e casistiche particolari.

11.3 Gestione con GitLab

GitLab è stato il nostro strumento centrale di collaborazione. Il workflow adottato:

- **Un branch per feature:** ogni funzionalità significativa veniva sviluppata su un branch dedicato, per evitare conflitti durante lo sviluppo parallelo.
- **Merge request con review:** prima di integrare nel branch principale, il codice veniva revisionato. Questo ha permesso di individuare bug e incongruenze prima che raggiungessero la versione stabile.
- **Commit descrittivi:** ogni commit include un messaggio che descrive chiaramente la modifica apportata.
- **Distribuzione del pacchetto:** la repository GitLab è stata utilizzata anche come canale di distribuzione del file APK per i test su dispositivi Android reali.

12. Valutazione del contributo di ciascun membro

Maltese Emanuel

Maltese Emanuel ha rappresentato una delle sorprese più positive del progetto. Lo avevo inizialmente sottovalutato, ma si è rivelato il mio principale punto di riferimento operativo, soprattutto durante il secondo e il terzo sprint. Ha sempre consegnato il proprio lavoro nei tempi stabiliti, con un'ottima qualità e una spiccata attenzione al refactoring, esattamente come avevo richiesto. Il suo contributo sulle schermate di inserimento manuale, sul dettaglio giornaliero e sui report grafici è stato determinante per la qualità finale dell'interfaccia.

Yu Serena

Serena ha dato un contributo creativo fondamentale al progetto. Si è occupata di buona parte del design, dei mockup e delle bozze dell'icona, imprimendo a NutriApp una coerenza visiva che si percepisce in tutta l'applicazione. Ha portato a termine tutti i compiti assegnati rispettando puntualmente le scadenze. La sua attenzione all'esperienza utente ha contribuito a rendere l'interfaccia intuitiva e piacevole da usare, un aspetto centrale per un'app di questo tipo.

Donzelli Christian

Christian ha gestito la documentazione fin dalle prime fasi del progetto, conducendo anche ricerche approfondite sulle API necessarie e occupandosi della loro implementazione. Il suo lavoro sulla documentazione LaTeX ha prodotto un documento completo e strutturato che copre aspetti tecnici, scientifici, legali e di distribuzione. È stato determinante per mantenere una visione d'insieme chiara e coerente del progetto, soprattutto nelle fasi finali. La sua competenza in ambito nutrizionale ha inoltre arricchito la qualità scientifica dell'applicazione.

Barakat Ismail (sottoscritto)

Mi sono occupato della direzione tecnica e organizzativa del progetto. Sul piano tecnico: sviluppo e manutenzione del backend PHP, progettazione e gestione del database MySQL/MariaDB, integrazione delle API esterne (OpenFoodFacts), testing delle funzionalità, gestione dei branch e dei merge su GitLab, assemblaggio finale del prodotto. Sul piano organizzativo: suddivisione e assegnazione dei compiti a inizio sprint, monitoraggio dell'avanzamento, refactoring sistematico del codice e formazione del team su Dart e Flutter.

Il ruolo dell'intelligenza artificiale

Il progetto si è avvalso del supporto dell'intelligenza artificiale per velocizzare le parti più ripetitive o meccaniche, tra cui la traduzione di testi, la revisione del codice e la verifica della correttezza sintattica. Si è rivelato uno strumento complementare utile, che ha accompagnato il team lungo tutto il percorso di sviluppo, accelerando alcune attività senza mai sostituire il ragionamento e le decisioni del team.

13. Sfide incontrate e insegnamenti

Sfide tecniche

- **Integrazione frontend-backend:** definire convenzioni condivise per la comunicazione tra Flutter e PHP (struttura del JSON, gestione degli errori, autenticazione tramite token) ha richiesto tempo e iterazioni.
- **Gestione dei molti campi nutrizionali:** tracciare macro e micronutrienti, fibre, acqua ed elettroliti ha richiesto uno schema del database complesso e una serializzazione JSON accurata, con attenzione costante alla coerenza dei calcoli.
- **Merge di branch e debugging:** lo sviluppo parallelo su branch separati ha generato conflitti che hanno richiesto debugging attento, soprattutto nella terza fase dello sprint.
- **Normalizzazione dei dati OpenFoodFacts:** l'API restituisce dati con struttura variabile a seconda del prodotto, richiedendo una logica di parsing robusta con gestione delle eccezioni.
- **Equilibrio tra semplicità e quantità di dati:** progettare un'interfaccia che mostri un numero elevato di nutrienti senza risultare sovraccarica è stata una delle sfide UX più difficili.
- **Test su iOS:** la compilazione e il test su iPhone senza licenza Apple Developer a pagamento ha richiesto l'utilizzo di AltServer + AltStore.

Insegnamenti principali

1. **Definire i modelli dati fin dall'inizio** riduce enormemente gli errori successivi e semplifica l'integrazione tra frontend e backend. Un campo rinominato nel database a progetto avanzato può richiedere modifiche a cascata in decine di file.
2. **Dividere il lavoro per moduli aiuta**, ma richiede convenzioni condivise forti e comunicazione costante. Senza un accordo su naming, struttura dei file e pattern di chiamata alle API, il codice diventa difficile da integrare.
3. **La documentazione aggiornata e lo storico Git sono fondamentali** per mantenere il controllo del progetto. In più occasioni, il diario di bordo e la storia dei commit ci hanno permesso di capire perché una scelta era stata fatta e come tornare a una versione stabile.

14. Sviluppi futuri

NutriApp ha una base tecnica solida e modulare che consente di implementare sviluppi futuri in modo strutturato. I principali miglioramenti previsti nella roadmap:

- **Ampliamento del database locale:** arricchire progressivamente il database interno con un maggior numero di alimenti, così da garantire risultati di ricerca più completi e affidabili
 - **Supporto multi-paese:** estendere la copertura geografica dell'app integrando database alimentari locali specifici per ogni paese, rendendo NutriApp utilizzabile in contesti culturali e gastronomici diversi.
 - **Avvisi e consigli contestuali:** introdurre un sistema di suggerimenti in tempo reale che analizzi i valori inseriti dall'utente e segnali eventuali squilibri, come un eccesso di zuccheri o sodio, proponendo alternative più equilibrate.
 - **Consulenza nutrizionale personalizzata tramite AI:** integrare un modello di intelligenza artificiale in grado di fornire consigli dietetici su misura, adattati alle esigenze, agli obiettivi e alle condizioni di salute specifiche di ciascun utente.
 - **Riconoscimento nutrizionale da foto:** sviluppare una funzionalità di computer vision che permetta all'utente di fotografare un pasto e ottenere automaticamente una stima dei valori nutrizionali, semplificando ulteriormente il tracciamento quotidiano.
-

15. Conclusione

Questo progetto è stato reso possibile dalla collaborazione attiva e costante di tutti i membri del team. È stata un'esperienza formativa sotto molti aspetti: abbiamo imparato un nuovo linguaggio di programmazione (Dart) e un nuovo framework (Flutter), abbiamo gestito un progetto complesso dall'analisi dei requisiti alla distribuzione, abbiamo lavorato in modo strutturato con metodologia Agile e abbiamo affrontato le sfide reali di un ambiente di sviluppo distribuito.

Siamo riusciti a creare un'applicazione che utilizzeremmo concretamente nella vita di tutti i giorni, raggiungendo tutti gli obiettivi che ci eravamo prefissati e andando oltre i requisiti iniziali della committente. NutriApp non è solo un prodotto sviluppato, ma anche una base tecnica solida su cui continuare a costruire.

Slide 1 — Copertina (1 min)

Nome app, logo, team, anno scolastico, committente.

Slide 2 — Chi siamo + Introduzione al progetto

Tabella con i 4 membri e i rispettivi ruoli (Project Manager, Frontend, UX/UI, Nutrition Expert).

Slide 3 — Il problema e l'obiettivo (2 min)

Perché esiste NutriApp? App esistenti hanno paywall, micronutrienti incompleti, UI caotica. Il vostro obiettivo: app gratuita, completa, user-friendly.

Slide 4 — Analisi della concorrenza (2 min)

Tabella comparativa: Yazio vs FatSecret vs NutriApp sui punti chiave (prezzo, micronutrienti, barcode, personalizzazione).

Slide 5 — I requisiti della committente (2 min)

Elenco visivo dei requisiti principali richiesti dalla Prof.ssa Mileo (tracking nutrienti, grafici, notifiche, multiplatforma, GDPR...).

Slide 6 — Stack tecnologico (2 min) → 2/3 slide spiegazione linguaggio con esempi

Tabella Flutter/Dart + PHP + MySQL + GitLab + OpenFoodFacts. Una riga per tecnologia, ruolo e motivazione.

Slide 7 — Perché Flutter (2 min)

Un codebase → Android + iOS. Rendering con Skia/Impeller. Hot Reload. Grafico o schema visivo della cross-compilation.

Slide 8 — Architettura del progetto (3 min)

Diagramma del flusso dati end-to-end: Flutter → ApiService → PHP backend → MySQL → risposta JSON → Riverpod → UI aggiornata.

Slide 9 — Metodologia Agile e sprint (2 min)

Timeline visiva dei 3 sprint con i macro-obiettivi di ciascuno. MoSCoW brevemente.

Slide 10 — Sprint 1: Analisi e progettazione (1 min)

Wireframe, mockup, formazione Dart/Flutter durante le vacanze natalizie, Product Backlog.

Slide 11 — Sprint 2: Funzionalità base (2 min)

Inserimento manuale, calcolo totali giornalieri, ricette personalizzate.
Screenshot o wireframe.

Slide 12 — Sprint 3: Funzionalità avanzate (2 min)

Barcode, login/registrazione, soglie personalizzate, grafici, notifiche push, report PDF.

Slide 13 — Funzionalità core vs aggiuntive (2 min)

Due colonne: requisiti soddisfatti  a sinistra, funzionalità extra aggiunte  a destra (dark mode, PDF export, tutorial in-app, privacy policy...).

Slide 14 — Integrazione OpenFoodFacts + barcode (2 min)

Schema del flusso: fotocamera → barcode_scan2 → endpoint PHP → OpenFoodFacts API → normalizzazione dati → form compilato. Limitazioni note (micronutrienti incompleti).

Slide 15 — Qualità del codice e GitLab (2 min)

Branch per feature, merge request con review, commit descrittivi, widget riutilizzabili, naming semantico, separazione responsabilità.

Slide 16 — DEMO LIVE (15 min)

SLdie in cui diciamo di passare alla dimostrazione

Slide 17 — Sfide e insegnamenti (2 min)

3-4 sfide chiave (integrazione frontend-backend, normalizzazione OpenFoodFacts, test iOS con AltStore) + 2-3 insegnamenti principali.

Slide 18 → valutazione membri

Slide 19 — Sviluppi futuri e conclusione (2 min)

Roadmap visiva con sviluppi futuri